



Effective data generation for imbalanced learning using conditional generative adversarial networks

Georgios Douzas, Fernando Bacao*

NOVA Information Management School, Universidade Nova de Lisboa, Portugal



ARTICLE INFO

Article history:

Received 1 June 2017

Revised 17 August 2017

Accepted 11 September 2017

Available online 13 September 2017

Keywords:

GAN
Imbalanced learning
Artificial data
Minority class

ABSTRACT

Learning from imbalanced datasets is a frequent but challenging task for standard classification algorithms. Although there are different strategies to address this problem, methods that generate artificial data for the minority class constitute a more general approach compared to algorithmic modifications. Standard oversampling methods are variations of the SMOTE algorithm, which generates synthetic samples along the line segment that joins minority class samples. Therefore, these approaches are based on local information, rather on the overall minority class distribution. Contrary to these algorithms, in this paper the conditional version of Generative Adversarial Networks (cGAN) is used to approximate the true data distribution and generate data for the minority class of various imbalanced datasets. The performance of cGAN is compared against multiple standard oversampling algorithms. We present empirical results that show a significant improvement in the quality of the generated data when cGAN is used as an oversampling algorithm.

© 2017 Elsevier Ltd. All rights reserved.

1. Introduction

Learning from imbalanced data is an important problem for the research community as well as the industry practitioners (Chawla, Japkowicz, & Kolcz, 2003). An imbalanced learning problem can be defined as a learning problem from a binary or multiple-class dataset where the number of instances for one of the classes, called the majority class, is significantly higher than the number of instances for the rest of the classes, called the minority classes (Chawla, Bowyer, Hall, & Kegelmeyer, 2002). The Imbalance Ratio (IR), defined as the ratio between the majority class and each of the minority classes, varies for different applications and for binary problems values between 100 and 100.000 have been observed (Chawla et al., 2002; Barua, Islam, Yao, & Murase, 2014).

Imbalanced data are a characteristic of multiple real-world applications such as medical diagnosis, information retrieval systems, fraud detection, detection of oil spills in radar images, direct marketing, automatic classification of land use and land cover in remote sensing images, detection of rare particles in experimental high-energy physics, telecommunications management and bioinformatics (Akbani, Kwek, & Japkowicz, 2004; He & Garcia, 2009; Clearwater & Stern, 1991; Graves et al., 2016; Verbeke, Dejaeger,

Martens, Hur, & Baesens, 2012; Zhao, Li, Chen, & Aihara, 2008). Standard learning methods perform poorly in imbalanced data sets as they induce a bias in favor of the majority class. Specifically, during the training of a standard classification method the minority classes contribute less to the minimization of the objective function. Also the distinction between noisy and minority class instances is often difficult. An important observation is that in many of these applications the misclassification cost of the minority classes is often higher than the misclassification cost of the majority class (Domingos, 1999; Ting, 2002). Therefore the methods that address the class imbalance problem aim to increase the classification accuracy for the minority classes.

There are three main approaches to deal with the class imbalanced problem (Fernández, López, Galar, Jesus, & Herrera, 2013). The first is the modification/creation of algorithms that reinforce the learning towards the minority class. The second approach is the application of cost-sensitive methods at the data or algorithmic level in order to minimize higher cost errors. The third and more general approach is the modification at the data level by rebalancing the class distribution through under-sampling, over-sampling or hybrid methods.

Our focus in this paper is oversampling techniques, which result in the generation of artificial data for the minority class. Standard oversampling methods are inspired by Synthetic Minority Oversampling Technique (SMOTE) algorithm (Chawla et al., 2002), generating synthetic samples along the line segment that joins minority class samples. A direct approach to the data generation process

* Corresponding author.

E-mail addresses: gdouzas@icloud.com (G. Douzas), bacao@novaims.unl.pt (F. Bacao).

would be the use of a generative model that captures the actual data distribution. Generative Adversarial Networks (GAN) is a recent method that uses neural networks to create generative models (Goodfellow et al., 2014). A conditional Generative Adversarial Network (cGAN) extends the GAN model by conditioning the training procedure on external information (Mirza & Osindero, 2014). In this paper we apply a cGAN on binary class imbalanced datasets, where the cGAN conditioning on external information are the class labels of the imbalanced datasets. The final generative model is used to create artificial data for the minority class i.e. the generator corresponds to an oversampling algorithm.

For the evaluation of cGAN as an oversampling method an experimental analysis is performed, based on 12 publicly available datasets from Machine Learning Repository. In order to test it on a wide range of IRs, additional datasets are created by undersampling the minority class of these 12 datasets as well as by adding simulated datasets with appropriate characteristics. Then the proposed method is compared to Random Oversampling, SMOTE algorithm, Borderline SMOTE (Han, Wang, & Mao, 2005), ADASYN (He, Bai, Garcia, & Li, 2008) and Cluster-SMOTE (Cieslak, Chawla, & Striegel, 2006). For the classification of the binary class data five classifiers and three evaluation metrics are applied.

The sections in the paper are organized as follows. In Section 2, an overview of related previous works and existing sampling methods is given. In Section 3, the theory behind GANs is described. Section 4 presents the proposed method in detail. Section 5 presents the research methodology. In Section 6 the experimental results are presented while conclusions are provided in Section 7.

2. Related work

Considering that our focus is the modification on the data level, and particularly the generation of artificial data, we provide a short review of the oversampling methods. A review of the other methods can be found in Galar, Fernández, Barrenechea, Bustince, and Herrera (2012) and Chawla (2005). Oversampling methods generate synthetic examples for the minority class and add them to the training set. A simple approach, known as Random Oversampling, creates new data by copying random minority class examples. The drawback of this approach is that the exact replication of training examples can lead to overfitting since the classifier is exposed to the same information.

An alternative approach that aims to avoid this problem is SMOTE. Synthetic data are generated along the line segment that joins minority class samples. SMOTE has the disadvantage that, since the separation between majority and minority class clusters is not often clear, noisy samples may be generated (He & Garcia, 2009). To avoid this scenario various modifications of SMOTE have been proposed. SMOTE+Edited Nearest Neighbor (Batista, Prati, & Monard, 2004) combination applies the edited nearest neighbor rule (Wilson, 1972) after the generation of artificial examples through SMOTE to remove any misclassified instances, based on the classification by its three nearest neighbors. Safe-Level SMOTE (Bunkhumpornpat, Sinapiromsaran, & Lursinsap, 2009) modifies the SMOTE algorithm by applying a weight degree, the safe level, in the data generation process. Borderline-SMOTE (Han et al., 2005), MWMOTE (Majority Weighted Minority Oversampling Technique for Imbalanced Data Set Learning) (Barua et al., 2014), ADASYN (He et al., 2008) and its variation KernelADASYN (Tang & He 2015) aim to avoid the generation of noisy samples by identifying the borderline instances of the majority and minority classes that in turn are used to identify the informative minority class samples.

The methods above address the problem of between-class imbalance (Nekooimehr & Lai-Yuen, 2016). Another type of prob-

lem is the within-class imbalance (Nekooimehr & Lai-Yuen, 2016; Bunkhumpornpat, Sinapiromsaran, & Lursinsap, 2012; Cieslak & Chawla, 2008; Jo & Japkowicz, 2004) i.e. when sparse or dense subclusters of minority or majority instances exist. Clustering based oversampling methods that deal with the between-class imbalance problem have recently been proposed. These methods are initially partitioning the input space and then apply sampling methods in order to adjust the size of the various clusters. Cluster-SMOTE applies the k-means algorithm and then generates artificial data by applying SMOTE in the clusters. Similarly DB-SMOTE (Bunkhumpornpat et al., 2012) uses the DB-SCAN algorithm to discover arbitrarily shaped clusters and generates synthetic instances along a shortest path from each minority class instance to a pseudo-centroid of the cluster. A-SUWO (Nekooimehr & Lai-Yuen, 2016) creates clusters of the minority class instances with a size, which is determined using cross validation and generates synthetic instances based on a proposed weighting system. SOMO (Douzas & Bacao, 2017) creates a two dimensional representation of the input space and based on it, applies the SMOTE procedure to generate intracluster and intercluster synthetic data that preserve the underlying manifold structure. Other types of oversampling approaches are based on ensemble methods (Wang, Minku, & Yao, 2015; Sun et al., 2015) such as SMOTEBoost (Chawla, Lazarevic, Hall, & Bowyer, 2003), DataBoost-IM (Guo & Viktor, 2004).

3. GAN and cGAN algorithms

In this section, we provide a summary of the GAN and cGAN frameworks following closely the notation in Goodfellow et al. (2014) and Gauthier (2015). The GAN is based on the idea of competition, in which a generator G and a discriminator D are trying to outsmart each other. The objective of the generator is to confuse the discriminator. The objective of the discriminator is to distinguish the instances coming from the generator and the instances coming from the original dataset. If the discriminator is able to identify easily the instances coming from the generator then, relative to its discrimination ability, the generator is producing low quality data. We can look at the GAN setup as a training environment for the generator where the discriminator, while also improving, is providing feedback about the quality of the generated instances, forcing the generator to increase its performance.

More formally, the generative model G , defined as $G: Z \rightarrow X$ where Z is the noise space of arbitrary dimension d_z that corresponds to a hyperparameter and X is the data space, aims to capture the data distribution. The discriminative model, defined as $D: X \rightarrow [0, 1]$, estimates the probability that a sample came from the data distribution rather than G . These two models, which are both multilayer perceptrons, compete in a two-player minmax game with value function:

$$\min_G \max_D V(D, G) = E_D + E_G$$

where:

$$E_D = E_{x \sim p_{data}(x)} [\log D(x)]$$

$$E_G = E_{z \sim p_z(z)} [\log (1 - D(G(z)))] \quad (1)$$

The $x \in X$ values are sampled from the data distribution $p_{data}(x)$ and the $z \in Z$ values are sampled from the noise distribution $p_z(z)$. The training procedure consists of alternating between k optimizing steps for D and one optimizing step for G by applying SGD. Therefore during training, D is optimized to correctly classify training data and samples generated from G , assigning 1 and 0 respectively. On the other hand the generator is optimized to confuse the discriminator by assigning the label 1 to samples generated from G . The unique solution of this adversarial game corresponds to G recovering the data distribution and D equal to $\frac{1}{2}$ for any input (Goodfellow et al., 2014).

The cGAN is an extension of the above GAN framework. An additional space Y is introduced, which represents the external information coming from the training data. The cGAN framework modifies the generative model G , to include the additional space Y , as follows:

$$G: Z \times Y \rightarrow X \quad (2)$$

Similarly to G the discriminator D is modified as follows:

$$D: X \times Y \rightarrow [0, 1] \quad (3)$$

Eq. (1) is also modified reflecting the redefinition of G and D :

$$\min_G \max_D V(D, G) = E_D + E_G$$

where:

$$E_D = E_{x, y \sim p_{data}(x, y)} [\log D(x, y)]$$

$$E_G = E_{z \sim p_z(z), y \sim p(y)} [\log (1 - D(G(z, y), y))] \quad (4)$$

The $(x, y) \in X \times Y$ values are sampled from the data distribution $p_{data}(x, y)$, the $z \in Z$ values are sampled from the noise distribution $p_z(z)$ and $y \in Y$ values are sampled from conditional data vectors found in the training data and represented by the density function $p_y(y)$. The training procedure for cGANs is identical to the training of the GAN model. Rephrasing Eq. (4), the cost functions for the gradient update of the discriminator and generator on a minibatch of m training examples $\{(x_i, y_i)\}_{i=1}^m$ and m noise samples $\{z_i\}_{i=1}^m$ are the following logistic cost expressions:

$$J_D = -\frac{1}{2m} \left(\sum_{i=1}^m \log D(x_i, y_i) + \sum_{i=1}^m \log(1 - D(G(z_i, y_i), y_i)) \right) \quad (5)$$

$$J_G = -\frac{1}{m} \sum_{i=1}^m \log D(G(z_i, y_i), y_i) \quad (6)$$

Eq. (6) is the modified version of the generator's loss function based on Eq. (4) in order to avoid saturation of the discriminator (Goodfellow et al., 2014). The cGAN model is trained by alternating gradient-based updates of Eqs. (5) and (6), similarly to the GAN framework, where D parameters are updated k times followed by a single update of G parameters.

4. The cGAN application to the imbalanced learning problem

The aim of the paper is to evaluate the effectiveness of a cGAN's generator G as an oversampling method. Specifically, the cGAN framework is trained on binary class imbalanced data $\{(x_i, y_i)\}_{i=1}^n$, where $(x_i, y_i) \in X \times \{0, 1\}$ with $y=1$ corresponding to the minority class. The external information of the CGAN process is represented by the class variable y . As it was mentioned above Z is a noise space of dimensionality d_z while the dimensionalities of X and Y are defined as d_x and d_y , respectively. Therefore the generator receives as input a vector that belongs to the $Z \times Y$ space and outputs a vector that belongs to the input space X . On the other hand the discriminator receives as input a vector that belongs to the $X \times Y$ space and classifies it as real or as, generated by G , artificial data. After the end of cGAN training the generator can be used to create artificial data for the minority class by receiving input vectors of the form $(z, y=1) \in Z \times Y$, where z is a sample from the noise space Z .

The hyperparameters of the above process are the dimension d_z of the noise space, the hyperparameters related to the G and D networks architecture as well as their training options. According to Eqs. (2) and (3), the networks architectures of the two models are constrained by the dimensionality of the input space X , the

Table 1
Description of the datasets. Comma separated values in the table cells correspond to different IR versions of the same dataset. Simulated datasets with labels 1 - 5 and 6-10 have a subclustering structure of 1 to 5 clusters for the minority class.

Dataset	Minority class	# Features	# Instances	# Minority instances	# Majority instances	Imbalance Ratio
Breast	car, fad	9	106, 88, 79	36, 18, 9	70	1.94, 3.89, 7.78
Ecoli	pp	7	336, 310, 297, 292	52, 26, 13, 8	284	5.46, 10.92, 21.85, 35.50
Glass	1	9	214, 179, 161, 155	70, 35, 17, 11	144	2.06, 4.11, 8.47, 13.09
Haberman	2	3	306, 265, 245, 238, 233	81, 40, 20, 13, 8	225	2.78, 5.62, 11.25, 17.31, 28.12
Heart	1	13	270, 210, 180, 170, 162, 158	120, 60, 30, 20, 12, 8	150	1.25, 2.50, 5.00, 7.50, 12.50, 18.75
Iris	2	4	150, 125, 112, 108	50, 25, 12, 8	100	2.00, 4.00, 8.33, 12.50
Libra	1, 2, 3	90	360, 324, 306, 300	72, 36, 18, 12	288	4.00, 8.00, 16.00, 24.00
Liver	1	6	345, 272, 236, 224, 214, 209	145, 72, 36, 24, 14, 9	200	1.38, 2.78, 5.56, 8.33, 14.29, 22.22
Pima	1	8	768, 634, 567, 544, 526, 517, 513	268, 134, 67, 44, 26, 17, 13	500	1.87, 3.73, 7.46, 11.36, 19.23, 29.41, 38.46
Segment	WINDOW	16	2310, 2145, 2062, 2035, 2013, 2001, 1996	330, 165, 82, 55, 33, 21, 16	1980	6.00, 12.00, 24.15, 36.00, 60.00, 94.29, 123.75
Vehicle	van	18	846, 746, 696, 680, 666, 660, 656	199, 99, 49, 33, 19, 13, 9	647	3.25, 6.54, 13.20, 19.61, 34.05, 49.77, 71.89
Wine	2	13	178, 142, 124, 118	71, 35, 17, 11	107	1.51, 3.06, 6.29, 9.73
Simulated1	1	20	4000	25	3975	159.00
Simulated2	1	20	4000	23	3977	172.91
Simulated3	1	20	4000	23	3977	172.91
Simulated4	1	20	4000	26	3974	152.85
Simulated5	1	20	4000	23	3977	172.91
Simulated6	1	200	3000	20	2980	149.00
Simulated7	1	200	3000	19	2981	156.89
Simulated8	1	200	3000	15	2985	199.00
Simulated9	1	200	3000	13	2987	229.77
Simulated10	1	200	3000	22	2978	135.36

Table 2

Dimension d_z and number of hidden units for G and D. Multiple numbers in the same cell correspond to the various IRs of Table 1.

Dataset	d_z	Number of units for hidden layer of G	Number of units for hidden layer of D
Breast	70, 70, 70	90, 90, 180	45, 45, 45
Ecoli	35, 55, 55, 45	40, 70, 70, 70	20, 35, 35, 30
Glass	25, 25, 35, 35	70, 70, 90, 90	35, 35, 45, 45
Haberman	10, 10, 12, 15, 15	20, 25, 25, 25, 25	10, 15, 15, 15, 15
Heart	65, 65, 65, 65, 65, 65	80, 80, 80, 80, 80, 80	40, 40, 40, 40, 40, 40
Iris	20, 20, 20, 20	25, 25, 25, 25	15, 15, 15, 15
Libra	180, 180, 180, 180	540, 540, 540, 540	270, 270, 270, 270
Liver	50, 50, 50, 25, 50, 50	35, 35, 35, 50, 50, 50	20, 20, 20, 20, 30, 30
Pima	15, 15, 15, 15, 40, 40, 40	50, 50, 50, 50, 25, 25, 25	80, 80, 80, 80, 80, 80, 80
Segment	80, 80, 80, 80, 80, 80, 80	160, 160, 160, 160, 160, 160, 160	80, 80, 80, 80, 80, 80, 80
Vehicle	70, 70, 70, 70, 90, 90, 90	100, 100, 100, 100, 100, 100, 100	55, 55, 55, 55, 55, 55, 55
Wine	130, 130, 130, 130	65, 65, 65, 65	40, 40, 40, 40
Simulated1	50	70	40
Simulated2	60	70	40
Simulated3	40	70	50
Simulated4	50	70	40
Simulated5	50	70	40
Simulated6	600	800	500
Simulated7	600	800	500
Simulated8	500	800	400
Simulated9	500	800	500
Simulated10	500	800	400

Table 3

Results for mean ranking of the oversampling methods across the datasets. The bold highlights the best performing method.

Metric	None	Random	SMOTE	Borderline SMOTE	ADASYN	Cluster-SMOTE	cGAN
<i>Algorithm: LR</i>							
AUC	4.37	3.79	3.54	4.42	4.89	4.15	2.85
F	5.31	3.99	3.69	3.34	4.20	4.92	2.56
G	6.73	2.94	3.18	3.82	3.14	5.69	2.49
<i>Algorithm: SVM</i>							
AUC	4.34	4.20	3.77	4.10	5.11	3.76	2.72
F	6.17	4.63	3.21	3.01	4.54	3.94	2.49
G	6.32	4.63	3.23	3.10	4.44	4.10	2.18
<i>Algorithm: KNN</i>							
AUC	5.01	5.37	3.42	3.70	4.27	3.72	2.51
F	5.87	3.90	3.62	3.32	4.07	4.70	2.51
G	6.59	4.63	2.59	3.77	2.93	5.25	2.23
<i>Algorithm: DT</i>							
AUC	5.23	4.76	3.70	4.04	3.61	3.96	2.70
F	5.34	4.10	3.76	4.03	3.68	4.28	2.82
G	5.46	5.18	3.44	3.92	3.27	4.18	2.55
<i>Algorithm: GBM</i>							
AUC	4.93	4.58	3.49	4.13	4.18	4.13	2.56
F	5.49	4.14	3.54	3.80	4.13	4.54	2.37
G	6.10	4.85	2.97	3.82	3.13	4.62	2.52

output space Y and the fact that D is a binary classifier. Specifically, the input and output layers of G have $d_z + d_y$ and d_x number of units, respectively. Also, the input and output layers of D have $d_x + d_y$ and one unit respectively. For a binary classification problem the dimensionality d_y of the class y is equal to one. Therefore, using a single hidden layer for G and D , the non-constrained hyperparameters of the cGAN are the dimension d_z of the noise space and the number of units for the hidden layers of G and D .

It is also important to notice that there are different formulations of the optimization problem as well as choices of the loss function of the cGAN framework. Following Goodfellow et al. (2014) and Gauthier (2015), we choose the vanilla cGAN formulation and the logistic cost functions given in Eqs. (5) and (6). The choice of the hyperparameters of the networks is described in the next section.

5. Research methodology

In order to test the performance of the proposed application of cGANs, 12 imbalanced datasets from the Machine Learning Reposi-

tory UCI were used. For each one of them, additional datasets were generated that resulted from undersampling the minority class of the initial datasets such that their final IR was increased approximately by a multiplication factor of 2, 4, 6, 10, 15 and 20. This procedure was applied for a given multiplication factor only when the final number of minority class instances was not less than 8. Additionally 10 artificial datasets were generated using the Python library Scikit-Learn (Pedregosa et al., 2011) that adapts an algorithm from Guyon (2003). Specifically, each class is composed of a number of gaussian clusters, in the range of 1 to 5, each located around the vertices of a hypercube. For each cluster, informative features are drawn independently from $N(0, 1)$ and then randomly linearly combined within each cluster in order to add covariance. The clusters are then placed on the vertices of the hypercube. Therefore a subcluster structure is created for the minority class. Table 1 shows a summary of the 71 data sets.

The performance of the cGAN as an oversampling method was evaluated and compared against Random Oversampling, SMOTE, Borderline SMOTE, ADASYN and Cluster-SMOTE. Since total accuracy is not appropriate for imbalanced datasets (balanced)

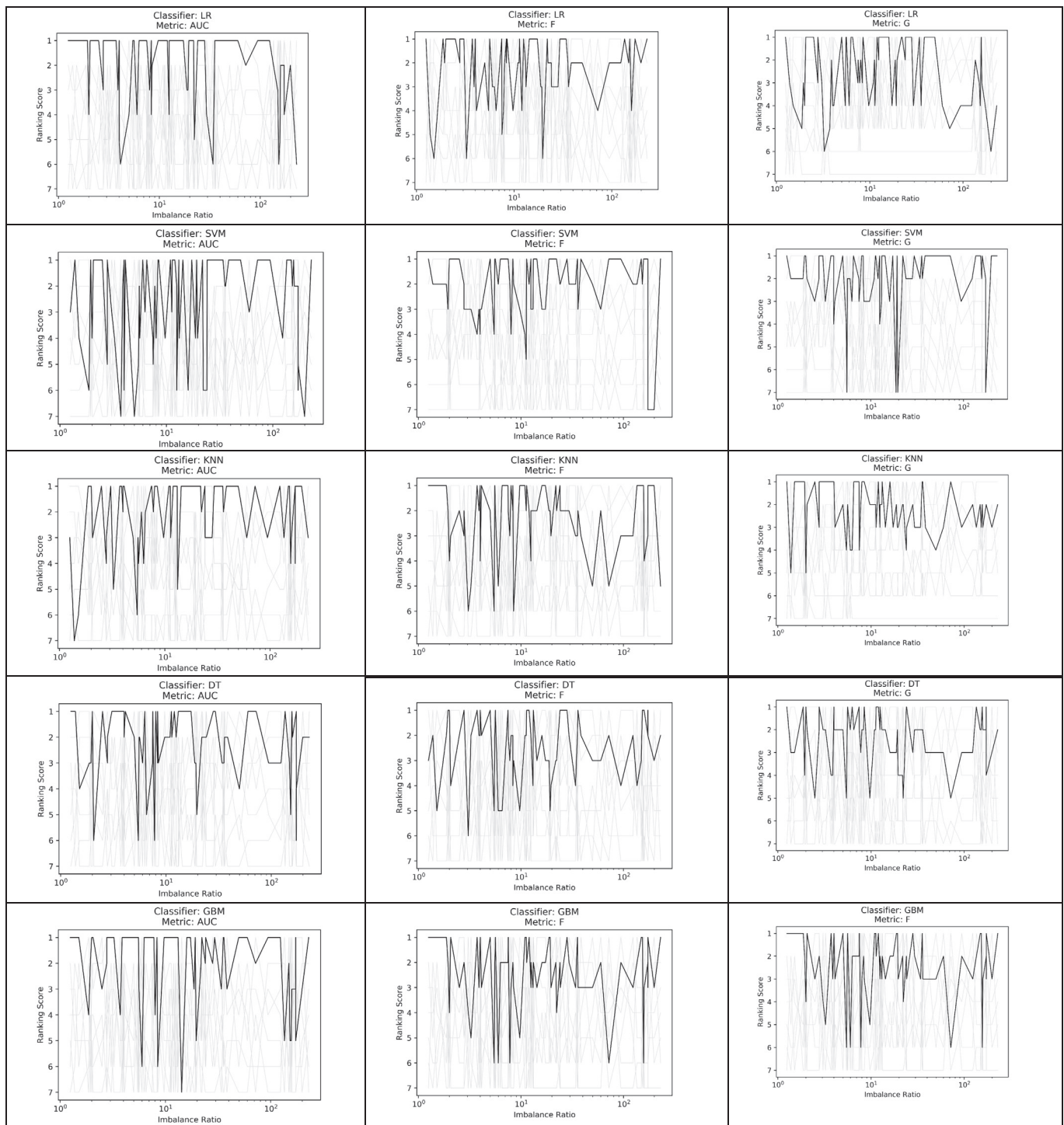


Fig. 1. Ranking of each oversampling method versus the datasets IRs. The bold polygon line correspond to the cGAN method.

F-measure, G-mean and Area Under the ROC Curve (AUC) are used (He & Garcia, 2009; Galar et al., 2012).

A ranking score was applied to each oversampling method for every combination of the 71 data sets, 3 evaluation metrics and 5 classifiers. Additionally to the 6 oversampling algorithms we also included the performance of the classifiers when no oversampling is used. Therefore the ranking score for the best performing method is 1 and for the worst performing method is 7. The Friedman test was applied to the ranking results, followed by the Holm's test where cGAN oversampler was considered as the control method (Guyon, 2003). Generally the Friedman test is used to

detect differences in the results across multiple experimental attempts, when the normality assumption may not hold. The null hypothesis is whether the classifiers have a similar performance across the oversampling methods and evaluation metrics when they are compared to their mean rankings. Holm's test is a non-parametric version of the *t*-test and the null hypothesis is whether the proposed application of cGAN outperforms the other methods as the control algorithm.

For the evaluation of the oversampling methods, Logistic Regression (LR) (McCullagh, 1984), Support Vector Machine with radial basis function (SVM) (Chang & Lin, 2011), Nearest Neighbors

Table 4
Results for Friedman's test.

Metric	p-value
<i>Algorithm: LR</i>	
AUC	2.46E–07
F	5.80E–15
G	6.06E–47
<i>Algorithm: SVM</i>	
AUC	1.21E–08
F	6.46E–28
G	1.34E–32
<i>Algorithm: KNN</i>	
AUC	1.50E–16
F	3.30E–20
G	1.60E–46
<i>Algorithm: DT</i>	
AUC	2.90E–11
F	1.57E–09
G	3.09E–19
<i>Algorithm: GBM</i>	
AUC	5.76E–10
F	7.40E–16
G	8.06E–29

(KNN) (Cover & Hart, 1967), Decision Trees (DT) (Quinlan, 1993) and Gradient Boosting Machine (GBM) (Friedman, 2001) were used. In order to evaluate the performance of the algorithms k-fold cross validation was applied with $k=5$. Before training, in each stage $i \in \{1, 2, \dots, k\}$ of the k-fold cross validation procedure, synthetic data $T_{g,i}$ were generated based on the training data T_i of the $k-1$ folds such that the resulting $T_{g,i} \cup T_i$ training set becomes perfectly balanced. This enhanced training set in turn was used to train the classifier. The performance evaluation of the classifiers was done on the validation data V_i of the remaining fold.

The hyperparameter tuning of the classifiers and the various oversampling algorithms was done using only the training data T_i of each cross validation stage $i \in \{1, 2, \dots, k\}$. Specifically, an additional training/validation splitting on T_i was applied as $T_i = T_{t,i} \cup T_{v,i}$. The classifiers were trained on $T_{t,i}$ and the optimal hyperparameters were selected in order to maximize the AUC of the validation set $T_{v,i}$. For SMOTE, Borderline SMOTE, Cluster-SMOTE and ADASYN algorithms the tuning parameters, for any combination of datasets, metrics and classifiers, were selected such that the performance of the classifier on the validation set $T_{v,i}$ is maximized when synthetic data were added to $T_{t,i}$ and formed the training set for the classifier.

A similar strategy was followed for the cGAN hyperparameter tuning. The validation performance was optimized only for the GBM classifier and the same cGAN hyperparameters were used for the rest of the classifiers. Furthermore instability issues during the cGAN training were observed. This problem was partially solved through the fine tuning of the hyperparameters, which provided a stable state. Both the generator and the discriminator used rectified linear units (Glorot, Bordes, & Bengio, 2011) as activation functions for the single hidden layer and sigmoid activations for the output layer. Each model was trained using the Adam optimizer (Kingma & Ba, 2015) with the default settings for both the initial learning rate and the exponential decay rates of the moment estimates. No dropout was applied to either the generator or the discriminator. Also the hyperparameter k , that controls the number of D parameters updates followed by a single G parameters update, was set equal to one since no improvement was observed for higher values. The optimal range for the number of epochs was found to be 2000–10,000 and the mini-batch size was set to the range from 1/20 up to 1/100 of each training set's sample size. Due to limited computational resources the search for the opti-

Table 5
Results for Holms' test. The bold highlights statistical significance.

		AUC		F		G
Adjusted a	Algorithm: LR					
0.0125	ADASYN	1.19e−08	Cluster-SMOTE	2.95e−15	None	5.62e−45
0.0143	Borderline SMOTE	1.21e−05	None	2.11e−12	Cluster-SMOTE	7.98e−35
0.0167	None	6.94e−05	ADASYN	9.55e−08	Borderline SMOTE	2.73e−07
0.0200	Cluster-SMOTE	1.68e−04	Random	1.51e−06	SMOTE	5.70e−03
0.0250	Random	3.94e−03	SMOTE	3.81e−05	ADASYN	1.59e−02
0.0333	SMOTE	3.75e−02	Borderline SMOTE	6.54e−03	Random	6.12e−02
Adjusted a	Algorithm: SVM					
0.0125	ADASYN	8.58e−11	None	1.06e−26	None	9.14e−41
0.0143	Random	1.20e−05	Random	2.67e−11	Random	4.72e−16
0.0167	None	2.85e−05	ADASYN	8.38e−09	ADASYN	1.25e−10
0.0200	Borderline SMOTE	4.29e−05	Cluster-SMOTE	1.66e−05	Cluster-SMOTE	3.08e−09
0.0250	Cluster-SMOTE	1.47e−03	SMOTE	8.83e−03	SMOTE	2.42e−05
0.0333	SMOTE	1.68e−03	Borderline SMOTE	5.70e−02	Borderline SMOTE	2.39e−04
Adjusted a	Algorithm: KNN					
0.0125	Random	2.11e−20	None	2.95e−19	None	7.95e−47
0.0143	None	7.47e−13	Cluster-SMOTE	9.18e−14	Cluster-SMOTE	4.06e−34
0.0167	ADASYN	2.74e−07	ADASYN	5.18e−07	Random	4.65e−20
0.0200	Cluster-SMOTE	1.41e−05	Random	1.28e−05	Borderline SMOTE	5.89e−12
0.0250	Borderline SMOTE	4.79e−05	SMOTE	3.67e−05	ADASYN	6.83e−03
0.0333	SMOTE	1.307e−03	Borderline SMOTE	3.06e−03	SMOTE	8.08e−02
Adjusted a	Algorithm: DT					
0.0125	None	1.63e−13	None	1.60e−12	None	1.22e−18
0.0143	Random	3.68e−10	Cluster-SMOTE	3.86e−08	Random	2.08e−17
0.0167	Cluster-SMOTE	7.76e−07	Borderline SMOTE	9.68e−05	Cluster-SMOTE	1.07e−12
0.0200	Borderline SMOTE	2.77e−05	Random	1.29e−04	Borderline SMOTE	1.49e−06
0.0250	SMOTE	7.20e−04	SMOTE	1.76e−03	SMOTE	5.96e−04
0.0333	ADASYN	5.67e−03	ADASYN	7.14e−03	ADASYN	1.34e−02
Adjusted a	Algorithm: GBM					
0.0125	None	9.83e−11	None	9.92e−18	None	1.12e−23
0.0143	Random	4.19e−09	Cluster-SMOTE	5.51e−15	Random	1.08e−13
0.0167	Cluster-SMOTE	3.01e−07	Random	2.10e−08	Cluster-SMOTE	2.24e−13
0.0200	Borderline SMOTE	1.66e−06	ADASYN	3.55e−08	Borderline SMOTE	9.82e−06
0.0250	ADASYN	2.77e−06	Borderline SMOTE	9.46e−07	ADASYN	5.12e−02
0.0333	SMOTE	2.39e−03	SMOTE	9.69e−06	SMOTE	7.49e−02

mal hyperparameters was not extensive. In fact it should be noted that the time and space complexity of the cGAN follows the standard neural network characteristics. Contrary to the other methods, the cGAN as an oversampler requires a training phase. Taking into consideration the size of the datasets used, standard oversampling methods were faster compared to the cGAN procedure. A mix of random grid search and manual selection was applied, choosing d_z values as well as the number of hidden layer's units for G and D to be a multiple of the dataset's number of features by a factor of 2 up to 10. The resulting optimal values are shown on Table 2.

The experimental procedure was repeated 5 times and the reported results include the average values between the experiments. The implementation of the classifiers and standard oversampling algorithms was based on the Python libraries Scikit-Learn (Pedregosa et al., 2011) and Imbalanced-Learn (Lemaitre, Nogueira, & Aridas, 2016). The cGAN implementation¹ is based on TensorFlow (Abadi et al., 2015).

6. Experimental results

The ranking results² are summarized using 15 plots. Each plot corresponds to a specific classifier and evaluation metric. The x-axis and y-axis of each plot represent the IR of the imbalanced dataset and the ranking score, respectively. Therefore each plot includes 7 polygon lines, corresponding to the 7 oversampling methods, where the bold line represents the cGAN oversampler (Fig. 1).

The mean ranking of the oversampling methods across the data sets for each combination of a classifier and evaluation metric is summarized in Table 3.

The cGAN method is the best performing method for all combinations of classification algorithms and evaluation metrics. In order to statistically confirm the conclusion, the Friedman test is applied and the results are shown in Table 4.

Therefore at a significance level of $\alpha=0.05$ the null hypothesis is rejected, i.e. the classifiers do not perform similarly in mean rankings for any evaluation metric. The Holm's test is applied with cGAN oversampler as the control method. The adjusted α and the p-values are shown in Table 5.

We observe that the cGAN oversampler outperforms all other methods for any evaluation metric when DT is used as a classifier. For the SVM, GBM and KNN classifiers, cGAN outperforms the other methods in 2 out of 3 evaluation metrics. Finally when LR is used as a classifier, cGAN has a higher performance compared to the other oversamplers in the F-measure. Although the simulated data have a subclustering structure for the minority class and vanilla cGAN suffers from the problem of mode collapse (Mehrjou & Saremi, 2017), where the generator focuses in parts of the low-dimensional data manifold, we did not observe a decrease in the performance of the classifiers in these cases.

7. Conclusions

In this paper we propose the use of cGAN, as an oversampling approach for binary class imbalanced data. The proposed application of cGAN results to a generative model G that has learned the actual data distribution conditioned on the class labels. G is used to generate synthetic data for the minority class i.e. as an oversampling algorithm. cGAN performance was evaluated on 71 datasets with different imbalance ratios, number of features and subclustering structures and compared to multiple oversampling methods, using Logistic Regression, Support Vector Machine, Nearest Neighbors, Decision Trees and Gradient Boosting Machine as

classifiers. The results show that cGAN performs better compared to the other methods for a variety of classifiers, evaluation metrics and datasets with complex structure. The explanation for this improvement in performance relates to the ability of cGAN to recover the training data distribution, if given enough capacity and training time. This ability is in contrast to standard oversampling methods where heuristic approaches are applied in order to generate minority class instances in safe areas of the input space. Although training the cGAN requires more effort and time compared to standard oversampling methods, once the training is finished, the generation of minority class instances is simple and effective. The generator accepts noise and the minority class label as input and outputs the generated data. Future research extensions of this work include the application of cGAN to the multiclass imbalanced problem and the search for more efficient ways to train G and D in the context of the imbalance learning problem.

References

- Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C. et al. (2015). TensorFlow: Large-scale machine learning on heterogeneous systems. Software available from tensorflow.org.
- Akbani, R., Kwek, S., & Japkowicz, N. (2004). Applying support vector machine to imbalanced datasets. *Machine Learning: ECML, 2004*, 39–50.
- Barua, S., Islam, Md. M., Yao, X., & Murase, K. (2014). MWMOTE - majority weighted minority oversampling technique for imbalanced data set learning. *IEEE Transactions on Knowledge and Data Engineering*, 26(2), 405–425.
- Batista, G., Prati, R., & Monard, M. (2004). A Study of the Behavior of Several Methods for Balancing Machine Learning Training Data. *ACM SIGKDD Explorations Newsletter - Special issue on learning from imbalanced datasets*, 6, 20–29.
- Bunkhumpornpat, C., Sinapiromsaran, K., & Lursinsap, C. (2009). Safe-Level-SMOTE: Safe-level-synthetic minority over-sampling technique for handling the class imbalanced problem. In *Lecture notes in computer science (including sub-series lecture notes in artificial intelligence and lecture notes in bioinformatics)* (pp. 475–482).
- Bunkhumpornpat, C., Sinapiromsaran, K., & Lursinsap, C. (2012). DBSMOTE: Density-based synthetic minority over-sampling technique. *Applied Intelligence*, 36(3), 664–684.
- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357.
- Chawla, N. V., Japkowicz, N., & Kolcz, A. (2003). Workshop learning from imbalanced data sets II. In *Proceedings of international conference on machine learning*.
- Chawla, N. V. (2005). Data mining for imbalanced data sets: an overview. In *Data mining and knowledge discovery handbook*. (pp. 875–886). Springer.
- Chawla, N. V., Lazarevic, A., Hall, L. O., & Bowyer, K. W. (2003). SMOTEBoost: Improving prediction of the minority class in boosting. In *Proceedings of seventh European conference principles and practice of knowledge discovery in databases* (pp. 107–119).
- Cieslak, D. A., & Chawla, N. V. (2008). Start globally, optimize locally, predict globally: Improving performance on imbalanced data. In *Proceedings - IEEE international conference on data mining, ICDM* (pp. 143–152).
- Clearwater, S. H., & Stern, E. G. (1991). A rule-learning program in high energy physics event classification. *Computer Physics Communications*, 67(2), 159–182.
- Cieslak, D. A., Chawla, N. V., & Striegel, A. (2006). Combating imbalance in network intrusion datasets. In *IEEE international conference on granular computation* (pp. 732–737).
- Cover, T. M., & Hart, P. E. (1967). Nearest neighbour pattern classification. *IEEE Transactions on Information Theory*, 13, 21–27.
- Domingos, P. (1999). MetaCost: A general method for making classifiers. In *Proceedings of the 5th international conference on knowledge discovery and data mining* (pp. 155–164).
- Douzas, G., & Bacao, F. (2017). Self-organizing map oversampling (SOMO) for imbalanced data set learning. *Expert Systems with Applications*, 82, 40–52.
- Fernández, A., López, V., Galar, M., Jesus, M. J., & Herrera, F. (2013). Analysing the classification of imbalanced data-sets with multiple classes: Binarization techniques and ad-hoc approaches. *Knowledge-Based Systems*, 42, 97–110.
- Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5), 1189–1232.
- Galar, M., Fernández, A., Barrenechea, E., Bustince, H., & Herrera, F. (2012). A review on ensembles for the class imbalance problem: Bagging, boosting, and hybrid-based approaches. *IEEE Transactions on Systems, Man and Cybernetics Part C: Applications and Reviews*, 42(4), 463–484.
- Gauthier, J. (2015). Conditional generative adversarial nets for convolutional face generation. Technical report.
- Chang, C., & Lin, C. (2011). LIBSVM: A library for support vector machines. *ACM Transactions on Intelligent Systems and Technology*, 2, 27.
- Glorot, X., Bordes, A., & Bengio, Y. (2011). Deep sparse rectifier neural networks. In *Proceedings of the fourteenth international conference on artificial intelligence and statistics, PMLR*: 15 (pp. 315–323).

¹ Code is available at <https://github.com/gdouzas/generative-adversarial-nets>

² The table format is available at <https://github.com/gdouzas/publications/blob/master/cGAN/results.csv>

- Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., et al. (2014). Generative adversarial nets. *Advances in Neural Information Processing Systems*, 27, 2672–2680.
- Graves, S. J., Asner, G. P., Martin, R. E., Anderson, C. B., Colgan, M. S., Kalantari, L., et al. (2016). Tree species abundance predictions in a tropical agricultural landscape with a supervised classification model and imbalanced data. *Remote Sensing*, 8, 161.
- Guo, H., & Viktor, H. (2004). Learning from imbalanced data sets with boosting and data generation: The DataBoost IM approach. *ACM SIGKDD Explorations Newsletter*, 6(1), 30–39.
- Guyon, I. (2003). Design of experiments for the NIPS 2003 variable selection benchmark.
- Han, H., Wang, W., & Mao, B. (2005). Borderline-SMOTE: A new over-sampling method in imbalanced data sets learning. *Advances In Intelligent Computing*, 17(12), 878–887.
- He, H., Bai, Y., Garcia, E., & Li, S. (2008). ADASYN: Adaptive synthetic sampling approach for imbalanced learning. In IEEE international joint conference on neural networks, 2008 (pp. 1322–1328).
- He, H., & Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9), 1263–1284.
- Jo, T., & Japkowicz, N. (2004). Class imbalances versus small disjuncts. *ACM Sigkdd Explorations Newsletter*, 6, 40–49.
- Kingma, D., & Ba, J. (2015). Adam: A method for stochastic optimization. *International conference on learning representations, ICLR*.
- Lemaitre, G., Nogueira, F., & Aridas, C. (2016). Imbalanced-learn: A python toolbox to tackle the curse of imbalanced datasets in machine learning. CoRR abs/1609.06570.
- McCullagh, P. (1984). Generalized linear models. *European Journal of Operational Research*, 16, 285–292.
- Mehrjou, A., & Saremi, S. (2017). Annealed generative adversarial networks. *International conference on learning representations, ICLR*.
- Mirza, M., & Osindero, S. (2014). Conditional generative adversarial nets. In *Proceedings of the neural information processing systems deep learning workshop (NIPS)*.
- Nekooeimehr, I., & Lai-Yuen, S. K. (2016). Adaptive semi-supervised weighted oversampling (A-SUWO) for imbalanced datasets. *Expert Systems with Applications*, 46, 405–416.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Quinlan, J. (1993). In *C4.5: programs for machine learning*: 16 (pp. 235–240). Morgan Kaufmann Publishers, Inc.
- Sun, Z., Song, Q., Zhu, X., Sun, H., Xu, B., & Zhou, Y. (2015). A novel ensemble method for classifying imbalanced data. *Pattern Recognition*, 48, 1623–1637.
- Tang, B., & He, H. (2015). KernelADASYN: Kernel based adaptive synthetic data generation for imbalanced learning. *IEEE Congress on Evolutionary Computation (CEC)*.
- Ting, K. M. (2002). An instance-weighting method to induce cost-sensitive trees. *IEEE Transactions on Knowledge and Data Engineering*, 14(3), 659–665.
- Verbeke, W., Dejaeger, K., Martens, D., Hur, J., & Baesens, B. (2012). New insights into churn prediction in the telecommunication sector: A profit driven data mining approach. *European Journal of Operational Research*, 218(1), 211–229.
- Wang, S., Minku, L. L., & Yao, X. (2015). Resampling-based ensemble methods for online class imbalance learning. *IEEE Transactions on Knowledge and Data Engineering*, 27(5), 1356–1368.
- Wilson, D. L. (1972). Asymptotic properties of nearest neighbor rules using edited data. *IEEE Transactions on Systems, Man and Cybernetics*, 2(3), 408–421.
- Zhao, X. M., Li, X., Chen, L., & Aihara, K. (2008). Protein classification with imbalanced data. *Proteins: Structure, Function and Genetics*, 70(4), 1125–1132.